Compte-rendu – MERN Séance 4

Authentification JWT, hachage Bcrypt et middlewares de sécurité

Mohamed Jegham – 5ème DS G3

École Polytechnique de Sousse – Dr. Abdelweheb GUEDDES – 2026–2027

---

1. Exercice 1 – Mise à jour sécurisée du mot de passe

La route `PUT /api/users/update-password` est protégée par `protect` : seul un utilisateur connecté peut changer **son propre** mot de passe, identifié par `req.user._id` (extrait du token, jamais du body).

Le middleware charge `req.user` avec `.select('-password')` : le hash n'y figure donc pas. Le contrôleur recharge l'utilisateur avec `User.findById(req.user._id)` pour récupérer le hash, puis `bcrypt.compare(oldPassword, user.password)` vérifie l'ancien mot de passe. En cas d'échec, la réponse est **401** « Ancien mot de passe incorrect ». Sinon, un **nouveau sel** est généré (`bcrypt.genSalt(10)`), `newPassword` est haché puis enregistré avec `user.save()`.

La longueur minimale (6 caractères) est vérifiée **avant** le hachage : le `minlength` du schéma s'appliquerait au hash (toujours 60 caractères) et ne bloquerait jamais un mot de passe trop court.

`controllers/userController.js – updatePassword`

```
// PUT /api/users/update-password (Exercice 1, protégé)
const updatePassword = async (req, res) => {
    try {
        const { oldPassword, newPassword } = req.body;

        if (!oldPassword || !newPassword) {
            return res.status(400).json({ message: 'oldPassword et newPassword sont requis'
});
        }

        if (newPassword.length < 6) {
            return res.status(400).json({
                message: 'Le nouveau mot de passe doit comporter au moins 6 caractères'
            });
        }

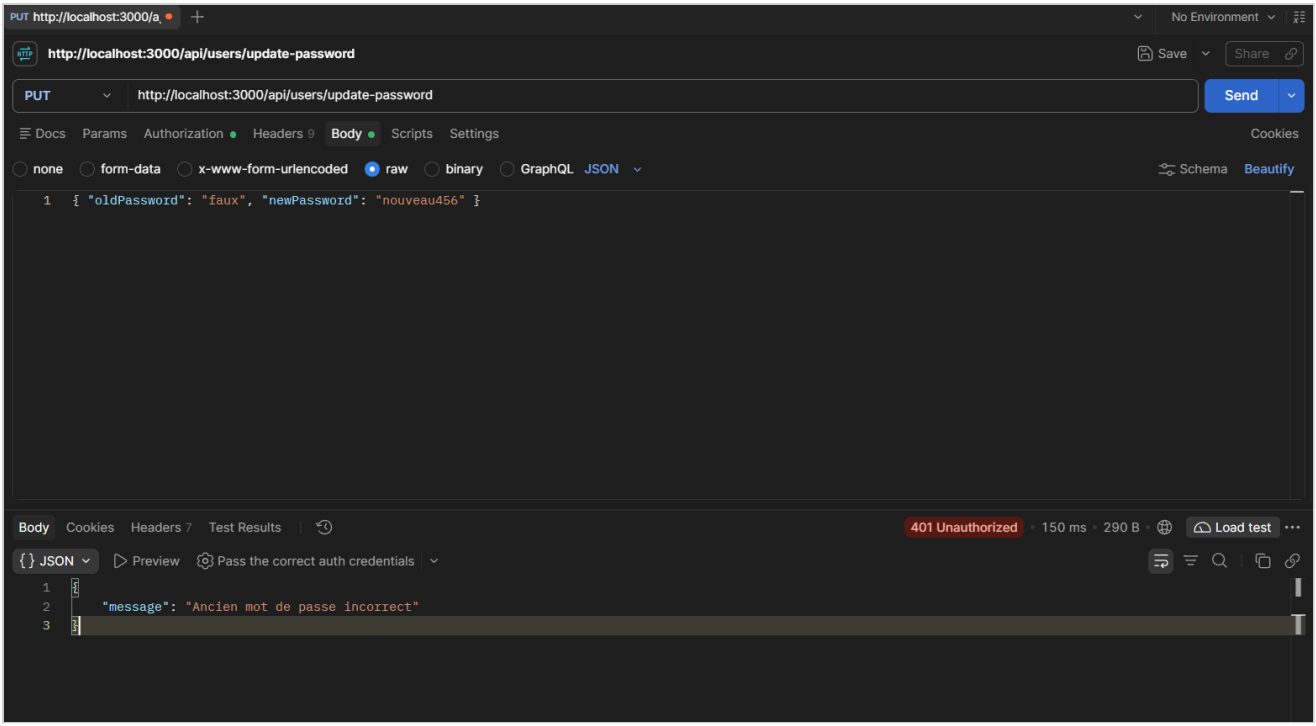
        // req.user a été chargé sans le mot de passe : on recharge le hash
        const user = await User.findById(req.user._id);

        const isMatch = await bcrypt.compare(oldPassword, user.password);
        if (!isMatch) {
            return res.status(401).json({ message: 'Ancien mot de passe incorrect' });
        }
```

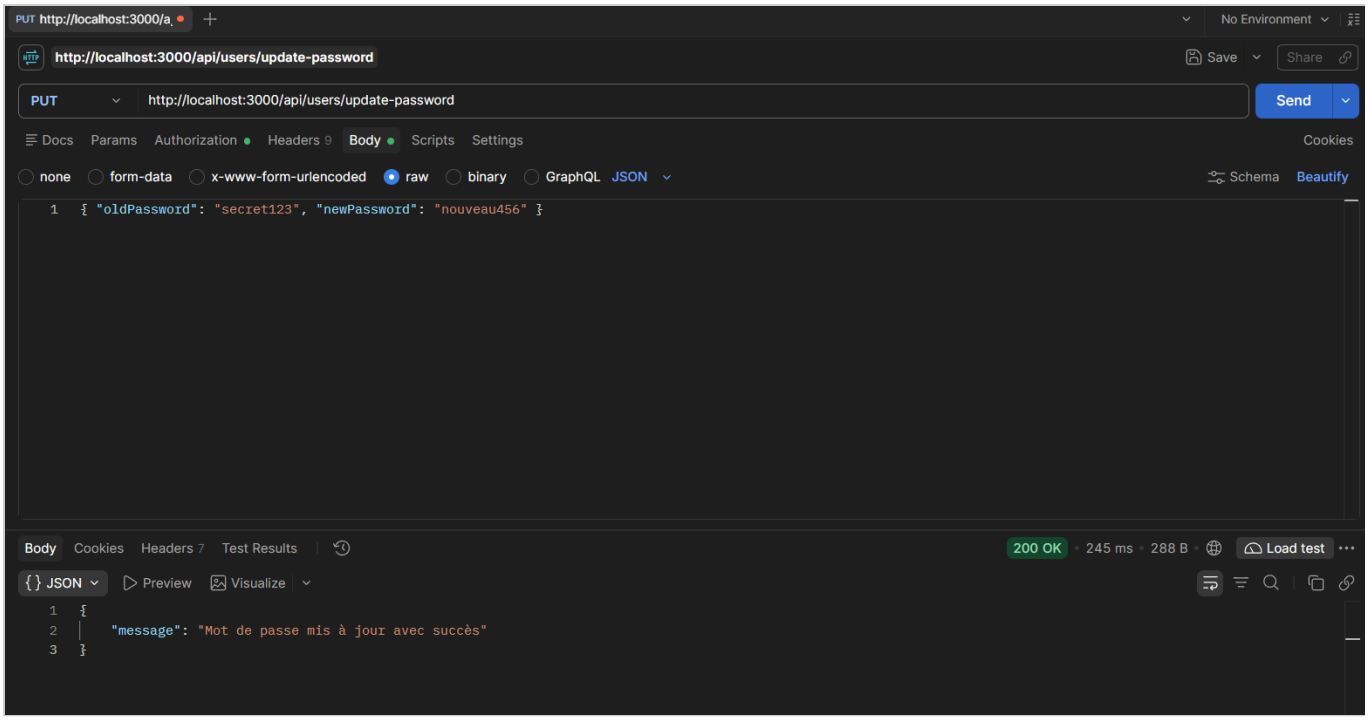
```
        const salt = await bcrypt.genSalt(10);
        user.password = await bcrypt.hash(newPassword, salt);
        await user.save();

        res.status(200).json({ message: 'Mot de passe mis à jour avec succès' });
    } catch (err) {
        gererErreur(err, res);
    }
};
```

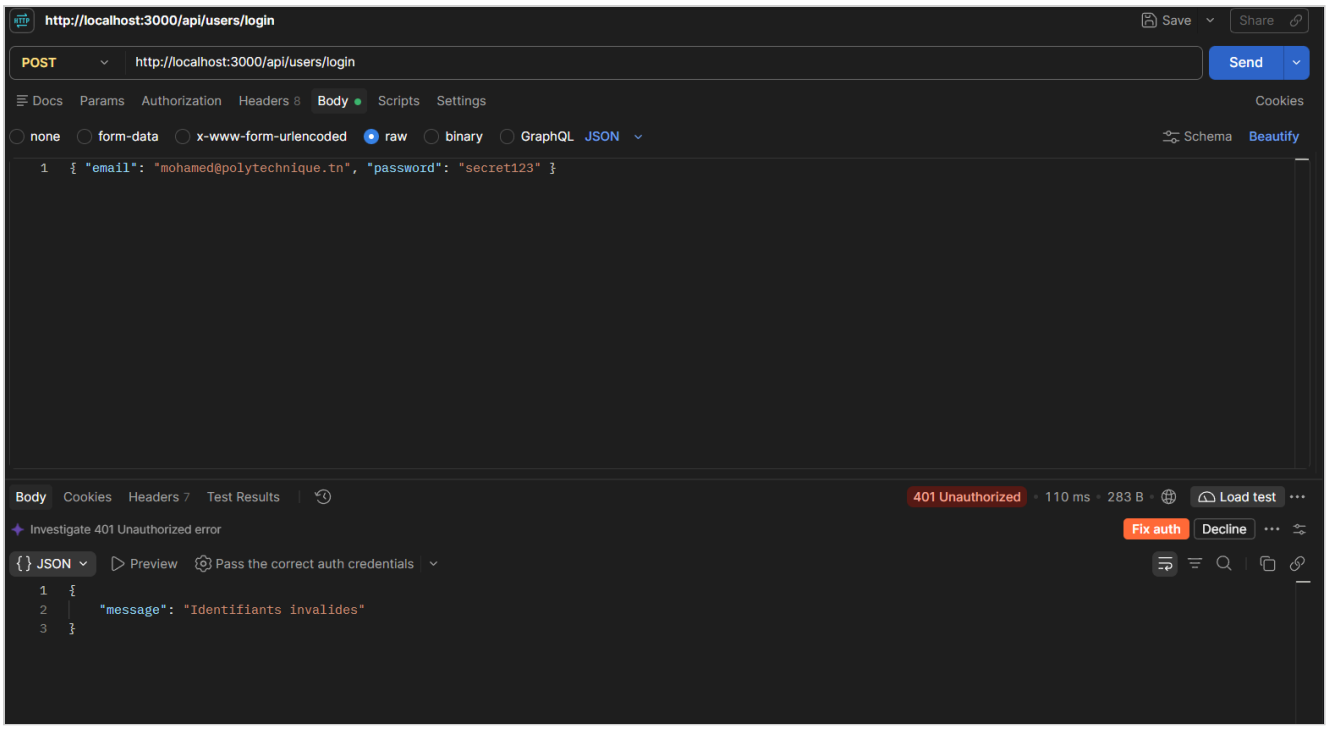
Requête	Cas	Statut
PUT /api/users/update-password	Sans token	401
PUT /api/users/update-password	Champ manquant ou `newPassword` < 6 caractères	400
PUT /api/users/update-password	`oldPassword` incorrect	401
PUT /api/users/update-password	`oldPassword` correct	200
POST /api/users/login	Ancien mot de passe après changement	401
POST /api/users/login	Nouveau mot de passe	200



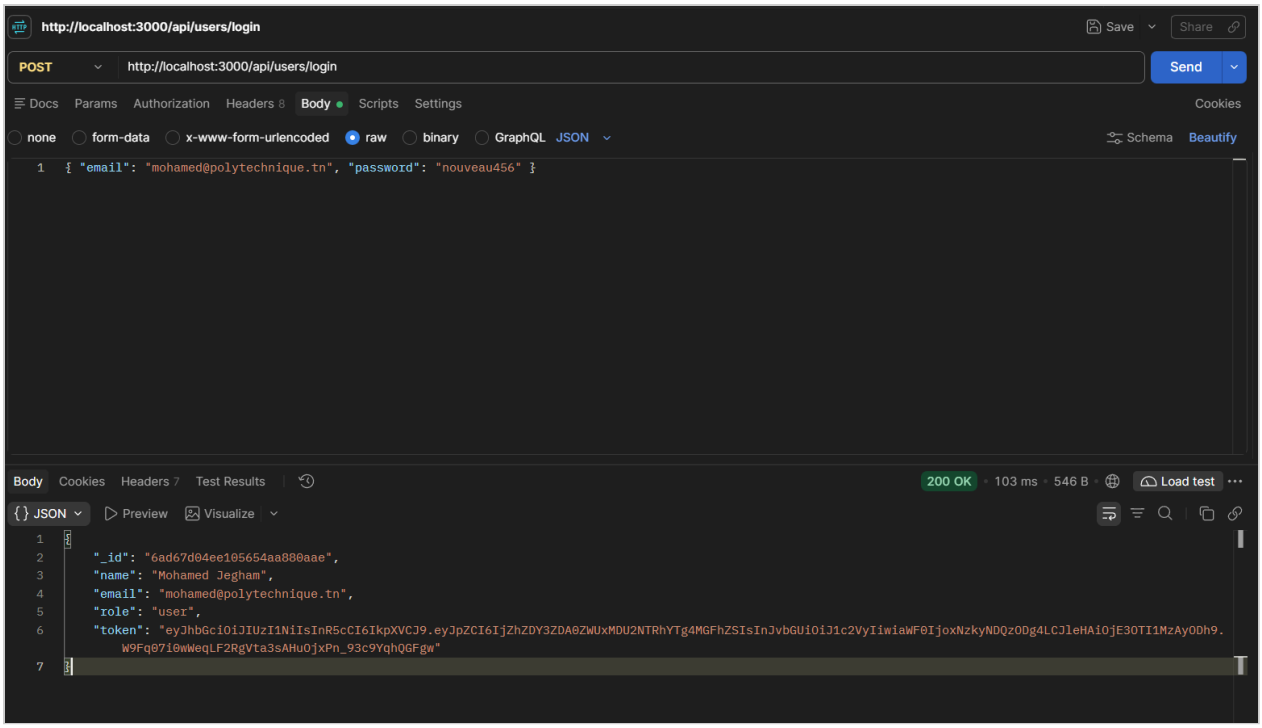
PUT /api/users/update-password avec un `oldPassword` erroné → **401 Unauthorized** « Ancien mot de passe incorrect ».



PUT /api/users/update-password avec le bon `oldPassword` → **200 OK**, mot de passe mis à jour.



POST /api/users/login avec l'**ancien** mot de passe → **401** « Identifiants invalides » : il n'est plus accepté.



POST /api/users/login avec le **nouveau** mot de passe → **200 OK** et nouveau token.

2. Exercice 2 – Articles personnels et attribution des rôles

2.1 Route GET /api/articles/my-articles

Express teste les routes **dans l'ordre de leur déclaration**. Si `/my-articles` était déclarée après `/:id`, la chaîne « my-articles » serait capturée comme un identifiant : `findById('my-articles')` lèverait une `CastError` (400). La route est donc placée avant `/:id` et protégée par `protect`. Le filtre `Article.find({ author: req.user._id })` utilise l'identité vérifiée du token.

controllers/articleController.js – getMyArticles

```
// GET /api/articles/my-articles (Exercice 2, protégé : articles de l'utilisateur connecté)
const getMyArticles = async (req, res) => {
    try {
        const articles = await Article.find({ author: req.user._id }).populate('author', 'name
email');
        res.status(200).json(articles);
    } catch (err) {
        gererErreur(err, res);
    }
};
```

routes/articleRoutes.js

```
const express = require('express');
const router = express.Router();
const articleController = require('../controllers/articleController');
const { protect } = require('../middlewares/authMiddleware');

// Consultation publique
router.get('/', articleController.getAllArticles);
```

```
// Exercice 2 : déclarée avant '/:id', sinon "my-articles" serait pris pour un id
router.get('/my-articles', protect, articleController.getMyArticles);

router.get('/:id', articleController.getArticleById);

// Opérations d'écriture protégées par authentification JWT
router.post('/', protect, articleController.createArticle);
router.put('/:id', protect, articleController.updateArticle);
router.delete('/:id', protect, articleController.deleteArticle);

module.exports = router;
```

2.2 Protection contre l'escalade de privilèges

Avant la correction, n'importe qui pouvait envoyer `"role": "admin"` à `/register` et obtenir tous les droits. Désormais, `role` n'est plus lu dans `req.body` et l'utilisateur est toujours créé avec `role: 'user'`.

controllers/userController.js – registerUser

```
// POST /api/users/register (Inscription)
const registerUser = async (req, res) => {
    try {
        // Exercice 2 : le champ role du body est ignoré (pas d'escalade de privilèges)
        const { name, email, password, age } = req.body;

        if (!name || !email || !password) {
            return res.status(400).json({ message: 'Nom, email et mot de passe requis' });
        }

        const userExists = await User.findOne({ email: email.toLowerCase().trim() });
        if (userExists) {
            return res.status(409).json({ message: 'Cet email est déjà utilisé' });
        }

        // Le minlength du schéma s'appliquerait au hash (60 caractères) : on le vérifie avant
hachage
        if (password.length < 6) {
            return res.status(400).json({
                message: 'Le mot de passe doit comporter au moins 6 caractères'
            });
        }

        // Hachage sécurisé avec sel (Salt factor = 10)
        const salt = await bcrypt.genSalt(10);
        const hashedPassword = await bcrypt.hash(password, salt);

        const user = await User.create({
            name,
            email,
            password: hashedPassword,
            age,
            role: 'user' // Inscription publique : toujours 'user'
        });
```

```
        res.status(201).json({
            _id: user._id,
            name: user.name,
            email: user.email,
            role: user.role,
            token: generateToken(user._id, user.role)
        });
    } catch (err) {
        gererErreur(err, res); // ValidationError -> 400, doublon -> 409
    }
};
```

2.3 Route d'administration PUT /api/users/:id/role

La route est protégée par `protect` puis `adminOnly` : un simple utilisateur reçoit **403**. Le rôle demandé est vérifié (`'user'` ou `'admin'`, sinon 400), puis appliqué avec `findByIdAndUpdate` et `runValidators: true`. La réponse exclut le mot de passe. L'inscription ne pouvant plus créer d'administrateur, le tout premier admin est promu directement dans MongoDB Atlas.

controllers/userController.js – updateUserRole

```
// PUT /api/users/:id/role (Exercice 2, admin uniquement)
const updateUserRole = async (req, res) => {
    try {
        const { role = 'admin' } = req.body;

        if (!['user', 'admin'].includes(role)) {
            return res.status(400).json({ message: "Le rôle doit être 'user' ou 'admin'" });
        }

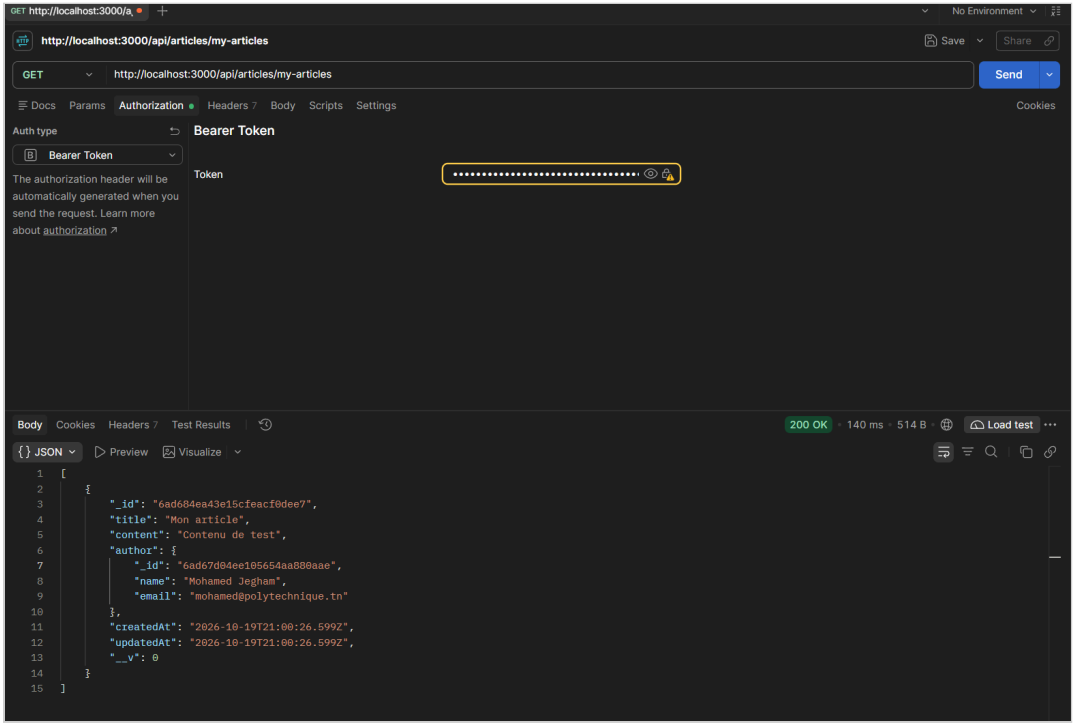
        const user = await User.findByIdAndUpdate(
            req.params.id,
            { role },
            { returnDocument: 'after', runValidators: true }
        ).select('-password');

        if (!user) {
            return res.status(404).json({ message: 'Utilisateur non trouvé' });
        }

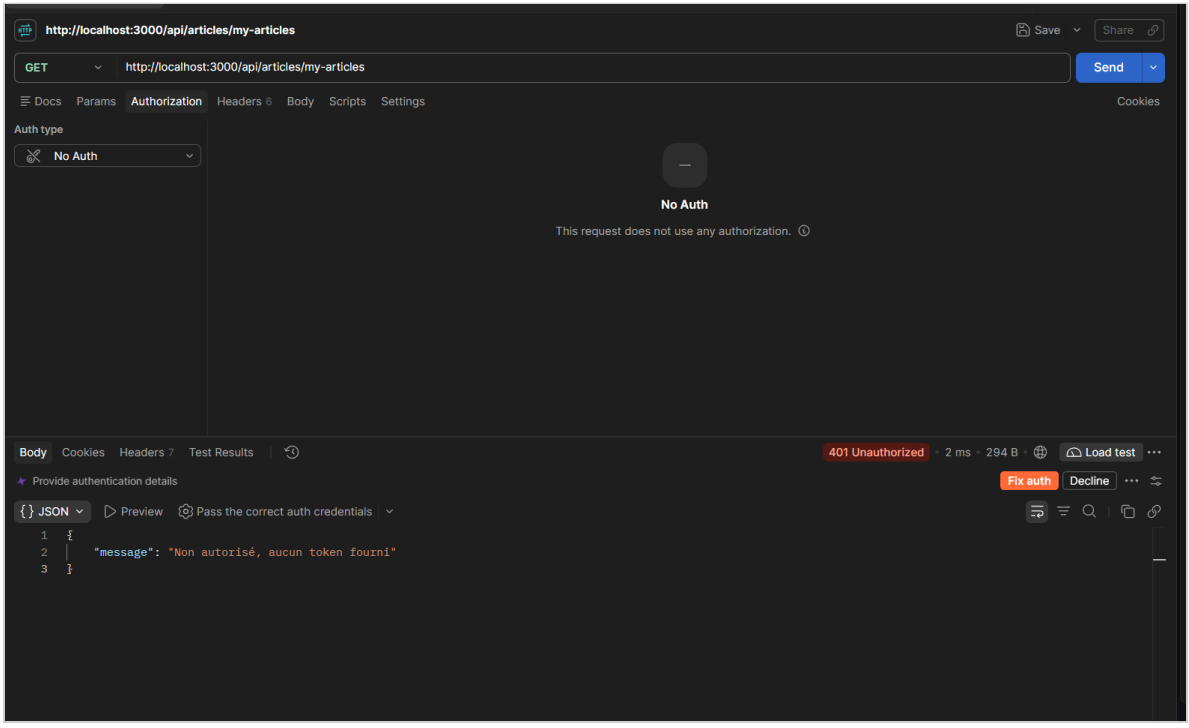
        res.status(200).json({ message: `Rôle mis à jour : ${user.role}`, user });
    } catch (err) {
        gererErreur(err, res); // CastError -> 400
    }
};
```

Requête	Cas	Statut
GET /api/articles/my-articles	Avec token	200 (seulement ses articles)
GET /api/articles/my-articles	Sans token	401
POST /api/users/register	`"role": "admin"` dans le body	201 avec `role: "user"`
PUT /api/users/:id/role	Token d'un utilisateur simple	403

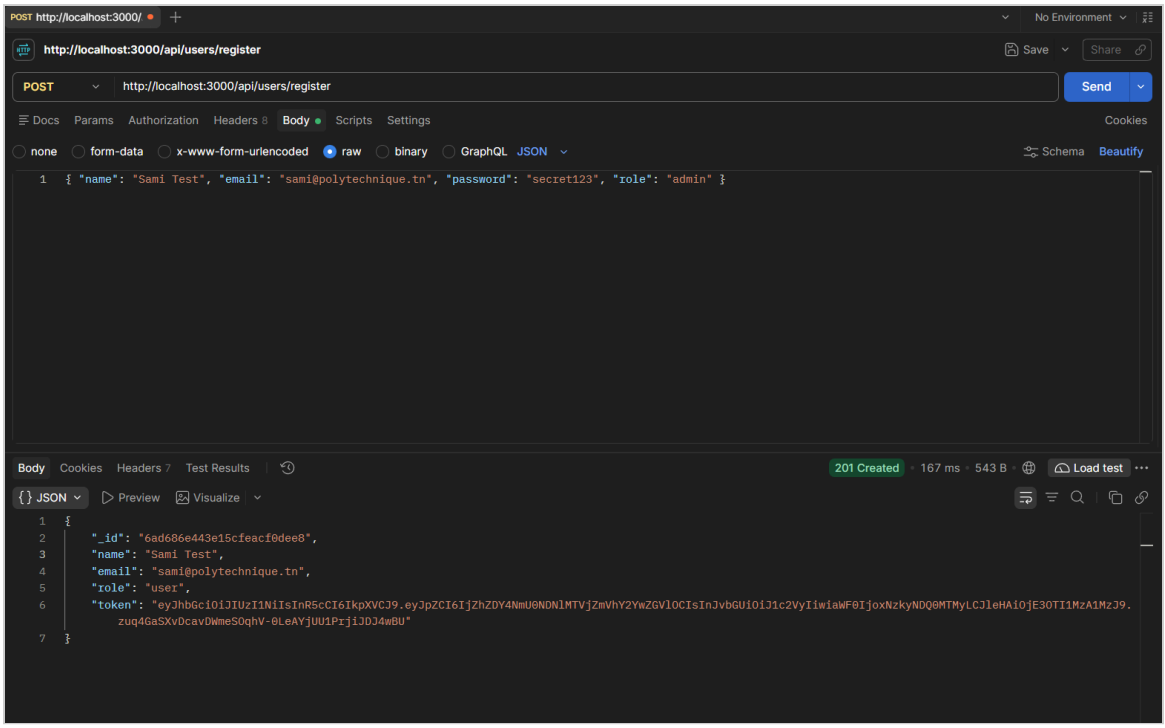
PUT /api/users/:id/role	Token admin, rôle invalide	400
PUT /api/users/:id/role	Token admin, `{ "role": "admin" }`	200



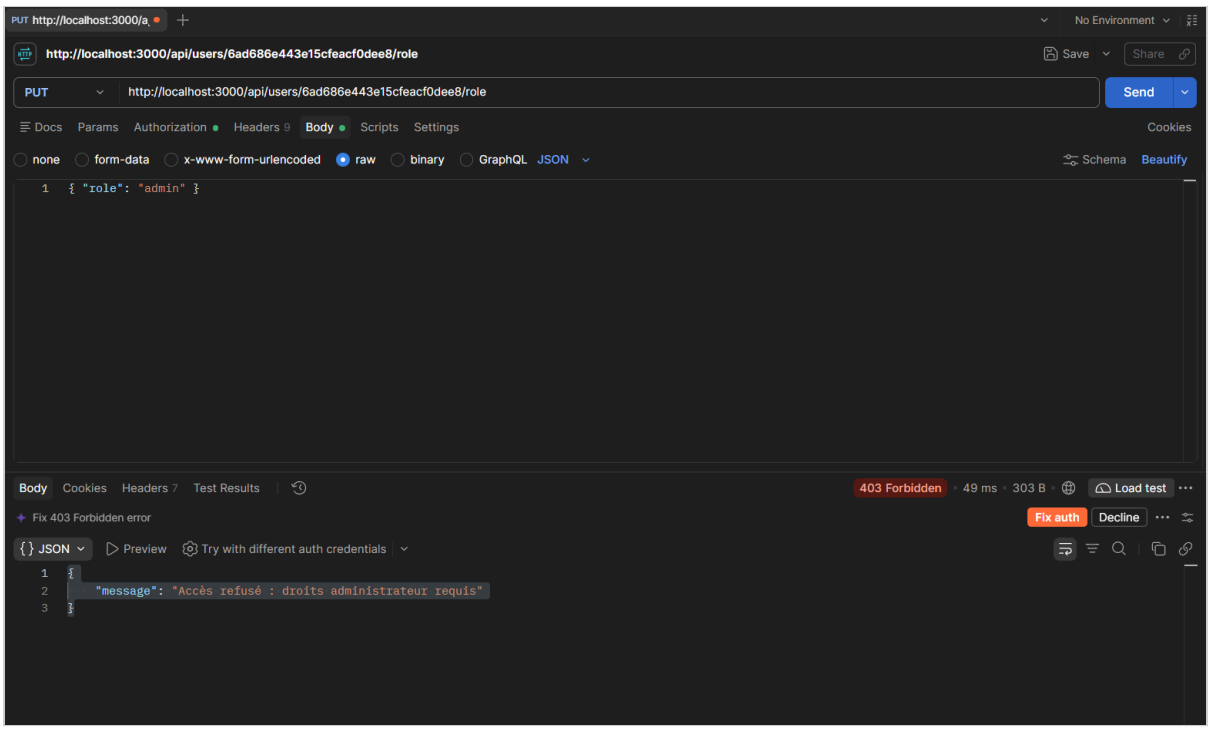
GET /api/articles/my-articles avec le token de Mohamed → **200 OK** : uniquement ses propres articles.



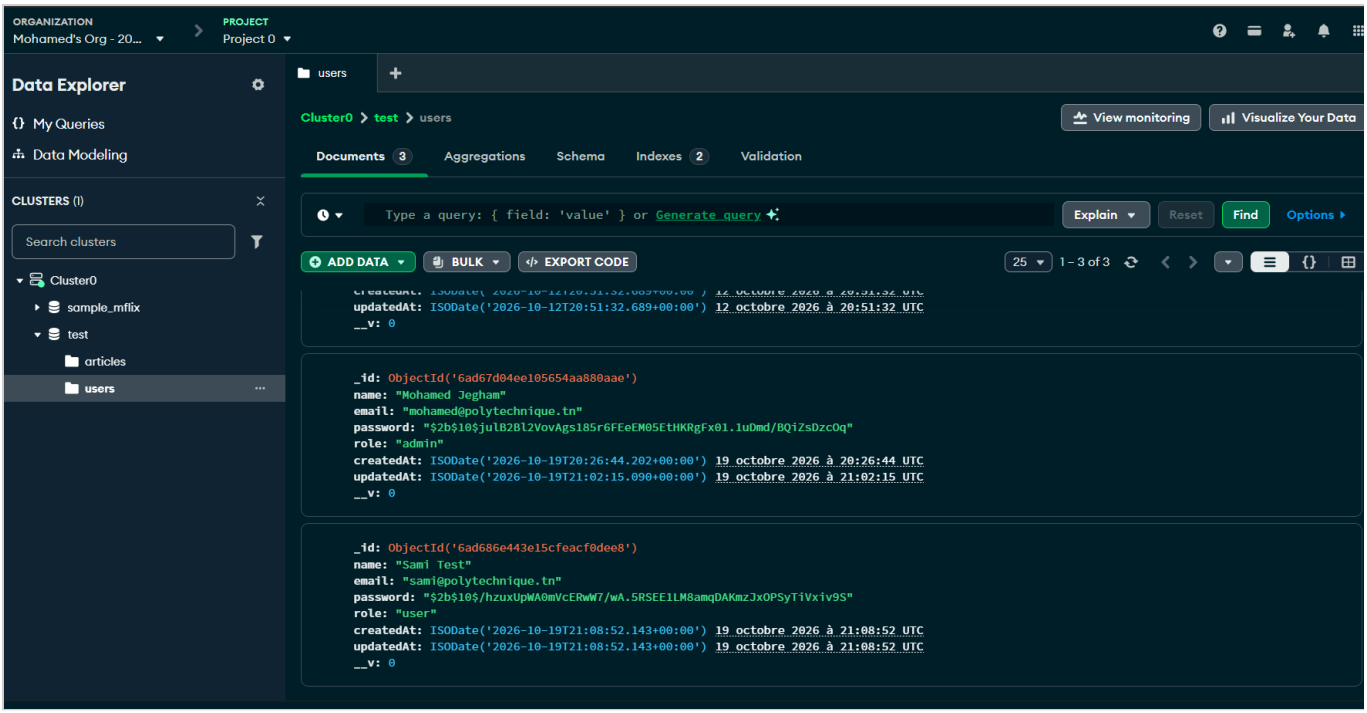
GET /api/articles/my-articles sans token → **401 Unauthorized** : la route est protégée.



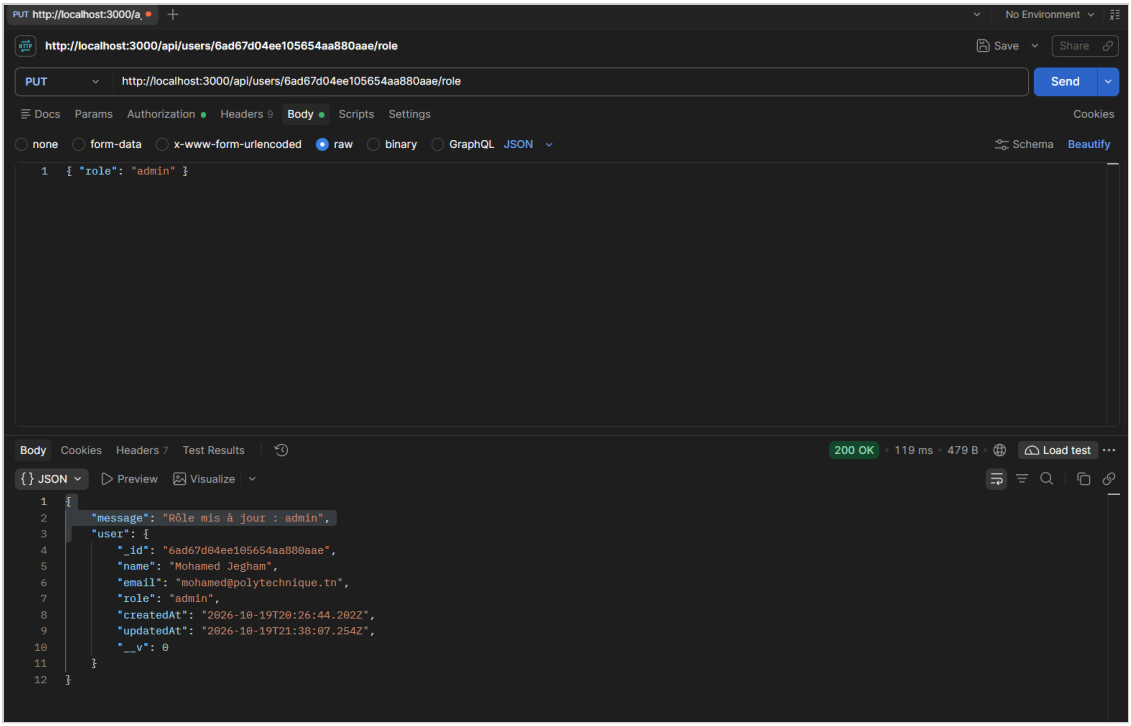
POST /api/users/register avec `"role": "admin"` dans le body → **201** mais `"role": "user"` : l'escalade de privilèges est bloquée.



PUT /api/users/:id/role avec le token d'un simple utilisateur → **403 Forbidden** (bloqué par `adminOnly`).



MongoDB Atlas : le premier administrateur est promu manuellement (`role: "admin"`), puisque l'inscription publique ne le permet plus.



PUT /api/users/:id/role avec le token administrateur → **200 OK**, l'utilisateur passe `admin`.

3. Exercice 3 – Gestion explicite de l'expiration du jeton

`jwt.verify()` lève une exception différente selon le problème : `TokenExpiredError` quand la date `exp` du payload est dépassée, `JsonWebTokenError` quand la signature ou le format est invalide. Le `catch` de `protect` teste `err.name === 'TokenExpiredError'` pour renvoyer un message précis. Le client (le futur front-end React) sait ainsi qu'il doit simplement renvoyer l'utilisateur vers la page de connexion, alors qu'un token falsifié garde le message générique.

middlewares/authMiddleware.js – protect

```
// 1. Protection des routes privées : vérifie la validité du JWT
const protect = async (req, res, next) => {
    let token;

    // Vérification de la présence de l'en-tête Authorization avec Bearer
    if (
        req.headers.authorization &&
        req.headers.authorization.startsWith('Bearer ')
    ) {
        try {
            // Extraction du token (après "Bearer ")
            token = req.headers.authorization.split(' ')[1];

            // Vérification cryptographique de la signature avec le secret
            const decoded = jwt.verify(token, process.env.JWT_SECRET);

            // Récupération de l'utilisateur associé (exclusion du mot de passe)
            req.user = await User.findById(decoded.id).select('-password');

            if (!req.user) {
                return res.status(401).json({
                    message: 'Non autorisé, utilisateur introuvable'
                });
            }

            // Passage au middleware ou contrôleur suivant
            return next();

        } catch (err) {
            // Exercice 3 : jeton expiré (expiresIn dépassé)
            if (err.name === 'TokenExpiredError') {
                return res.status(401).json({
                    message: 'Votre session a expiré, veuillez vous reconnecter'
                });
            }

            return res.status(401).json({
                message: 'Non autorisé, token invalide ou expiré'
            });
        }
    }

    // Si aucun en-tête Authorization n'est fourni
    return res.status(401).json({
        message: 'Non autorisé, aucun token fourni'
    });
};
```

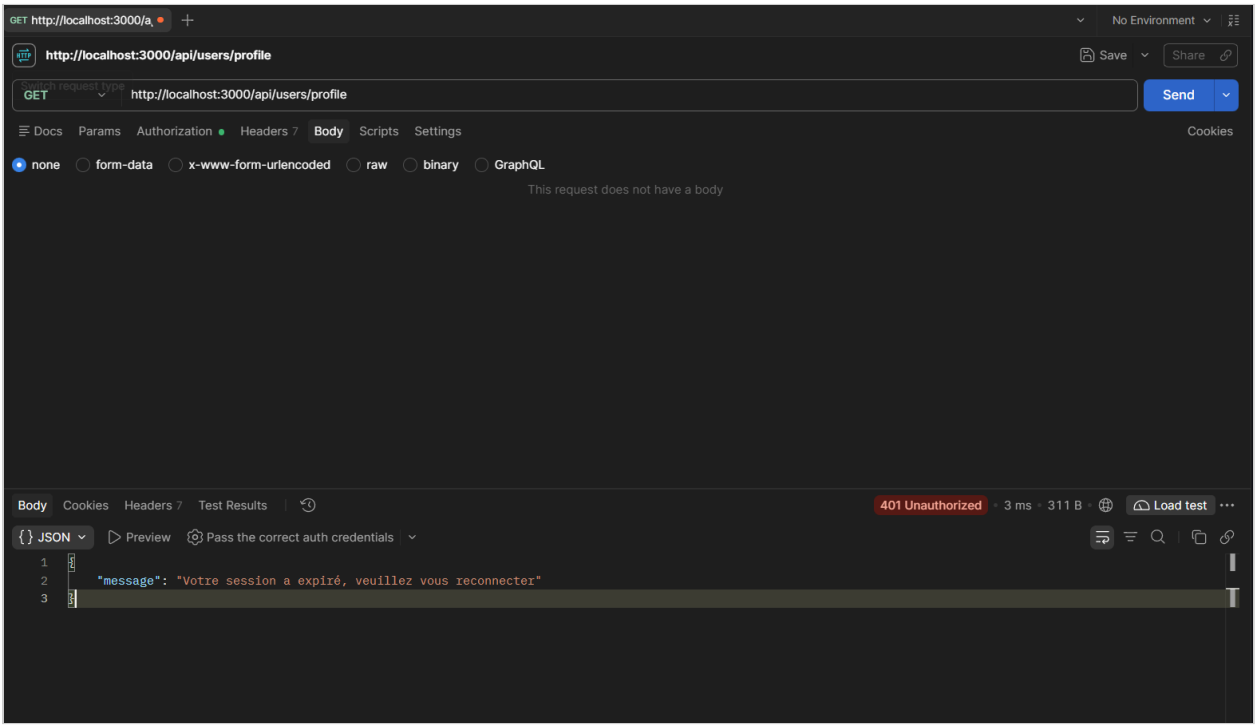
Test : `expiresIn` passé temporairement à `'10s'` dans `generateToken`, connexion, attente de 15 secondes, puis appel de `GET /api/users/profile`. La durée a ensuite été remise à `'1d'`.

Token envoyé à GET /api/users/profile	Statut	Message

Valide	200	Profil de l'utilisateur
Expiré	401	Votre session a expiré, veuillez vous reconnecter
Signature modifiée	401	Non autorisé, token invalide ou expiré



Modification temporaire de `generateToken` : `expiresIn: '10s'`.



GET /api/users/profile avec un token de plus de 10 secondes → **401** « Votre session a expiré, veuillez vous reconnecter ».